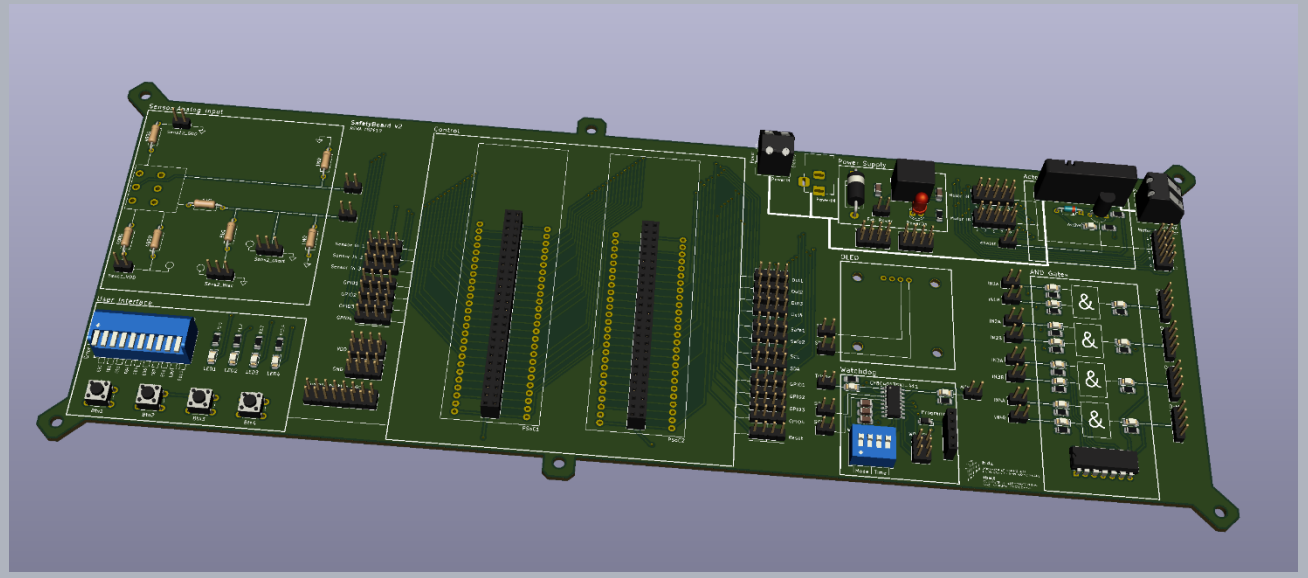
# Safety in Embedded Control Systems

## Qualification

Prof. Dr.-Ing. Peter Fromm



# Content

- How good is good enough?
- Reviews
- Static Code Analysis
- Metrics
- Tests
- Timing Analysis
- Tool Qualification

# Final System: Qualitative System Verification

| Id | System Requirements | $ASIL_{req}$ | HW / SW Requirement | Status |
|---|---|---|---|---|
| SR1 | The brake signal must be reliable | C | • SW: Check brake pedal operation power on<br>• SW: Ensure reliable reading of "pressed state“<br>• HW: Provide 1oo2 brake signal<br>• HW: Provide diagnostics to detect short circuits / broken lines of brake pedal |  |
| SR2 | The gas pedal signal must be reliable | C | • HW: Add resistors to detect broken / short circuit connections of gas pedal<br>• HW: Provide 1oo2 pedal signal<br>• HW: Use different technologies to read both channels<br>• HW: Use inverted signals<br>• SW: Drift of potentiometer must be detected<br>• SW: Short circuits / broken lines of gas pedal must be detected by complex device driver |  |
| SR3 | The car may only start after an explicit ignition sequence | QM | • SW: A clear press of the ignition button must be detected<br>• SW: The engine may only start of the gas pedal is depressed |  |
| SR4 | The control speed value must be correctly calculated | C | • SW: Brake signal must overide gas pedal<br>• SW: Wrong calculations of control speed value must be detected<br>• SW: Supervise / Limit PID Control Parameters<br>• SW: Driving direction may only be changed if engine is stopped<br>• SW: Reliable monitor communication link<br>• SW: Controlspeed will be calculated with different algo's on both controllers<br>• SW: State Machine protection |  |

# Final System: Qualitative System Verification

| Id | System Requirements | $ASIL_{req}$ | HW / SW Requirement | Status |
|---|---|---|---|---|
| SR5 | The engine currentspeed must be in line with the controlspeed value | C | • HW: Engine power must be deactivated in case of critical failure<br>• HW: Engine power must be controlled by independent hardware<br>• HW: Engine current must be supervised and limited<br>• SW: Engine Blocking must be detected<br>• SW: Current speed must be compared with target speed<br>• SW: Limit reverse speed<br>• SW: Engine must be explicitly activated |  |
| SR6 | The system status must be shown to the driver | A | • SW: Provide status / error information to driver<br>• SW: Detect freeze of OLED display |  |
| SR7 | The system integrity must be supervised | C | • HW: Provide an external watchdog, which turns of the actuator in case of a critical error<br>• HW: Power supply must be supervised, bursts must be avoided<br>• HW: The integrity of the controller (MCU) must be supervised<br>• SW: The integrity of the OS must be supervised<br>• SW: ISR bursts must be detected<br>• SW: Low Power must be detected |  |

# Implemented Safety Concepts

Mitigation of sensor faults
- Detection of short circuits and broken connections using resistors
- Detection of wear-off using redundant potentiometers
- Self-test of brake pedal

Mitigation of user faults
- Long press of ignition button
- Change of direction only in case of stand still

Mitigation of actor faults
- Robustness clippers in engine controller
- Comparison of PID result with open loop calibration curve

Mitigation of logic and system faults
- Alive Watchdog (external / internal)
- Second controller act as monitor

# Main Requirements for a Safety Architecture

Avoid Dangerous Failures

Reliable detection and handling of stochastic faults and errors (focus single faults)
E.g. hardware failures, communication errors,…

Reduction / avoidance of systematic errors.
E.g. programming bugs

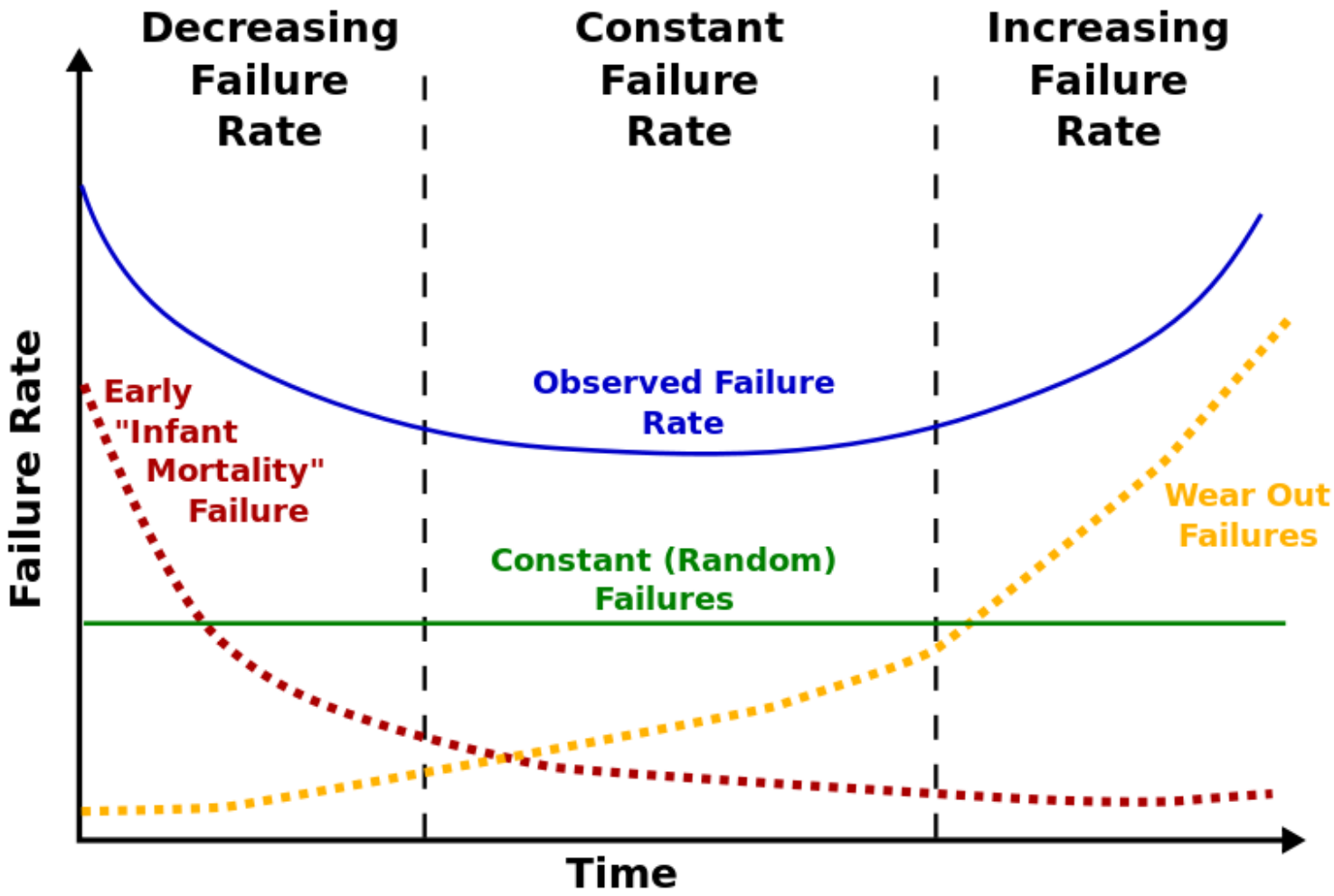

In case of a critical failure: Transition into a safe state.

# In code we trust…

Sure!

Did you test your code?

# Bathtub Curve – Hardware versus Software



# Which problems could still persist?

System Level

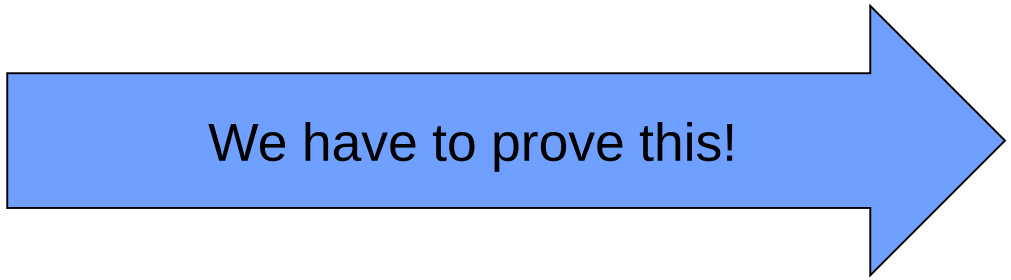
- First of all – did we really find and address all safety requirements?

Hardware

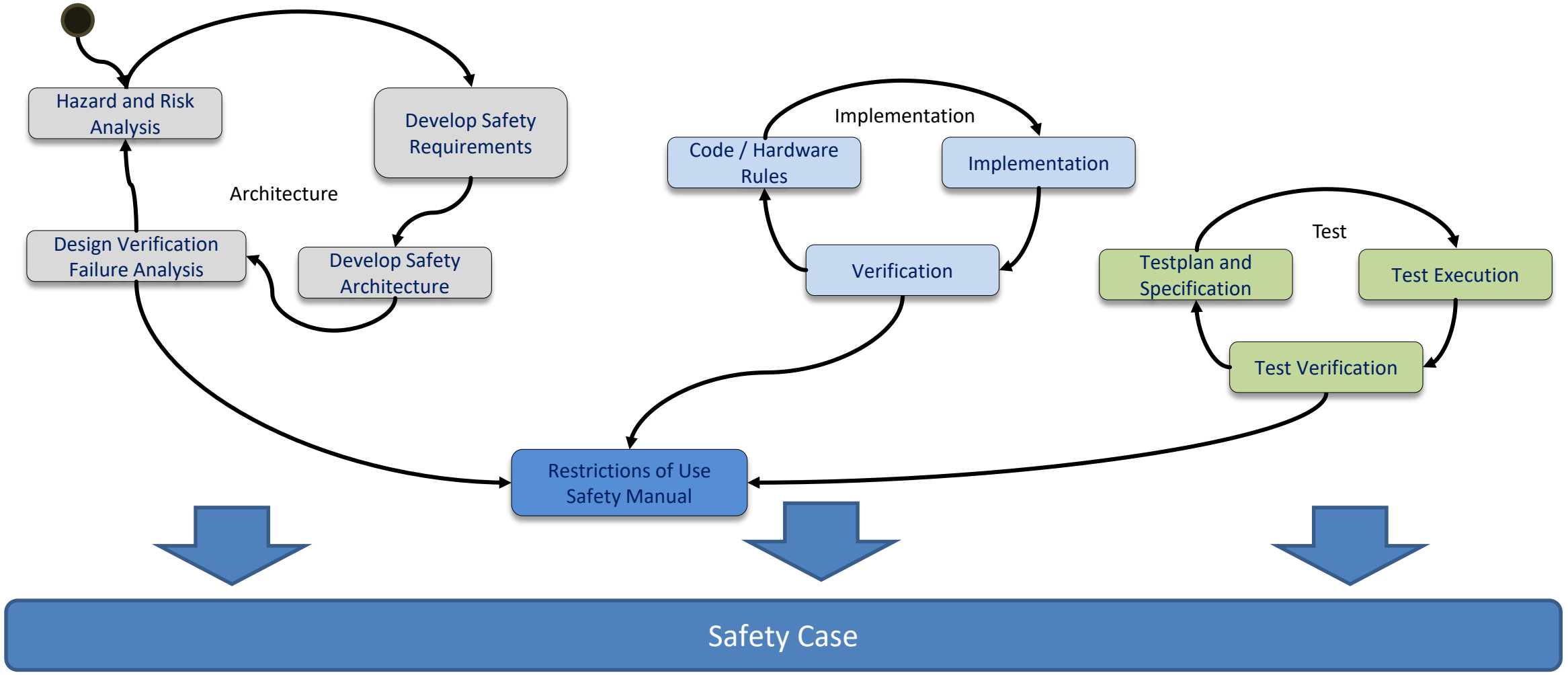
- Did we identify and handle all single point faults?
- For higher ASIL levels (C/D) – did we also consider all relevant dual point faults?

Software

- Did we specify sufficient tests for our safety functions?
- Did we cover all safety functions with our testcases?
- Did we perform sufficient stresstests and long time integration and system tests?



# Proof: Our Safety Case

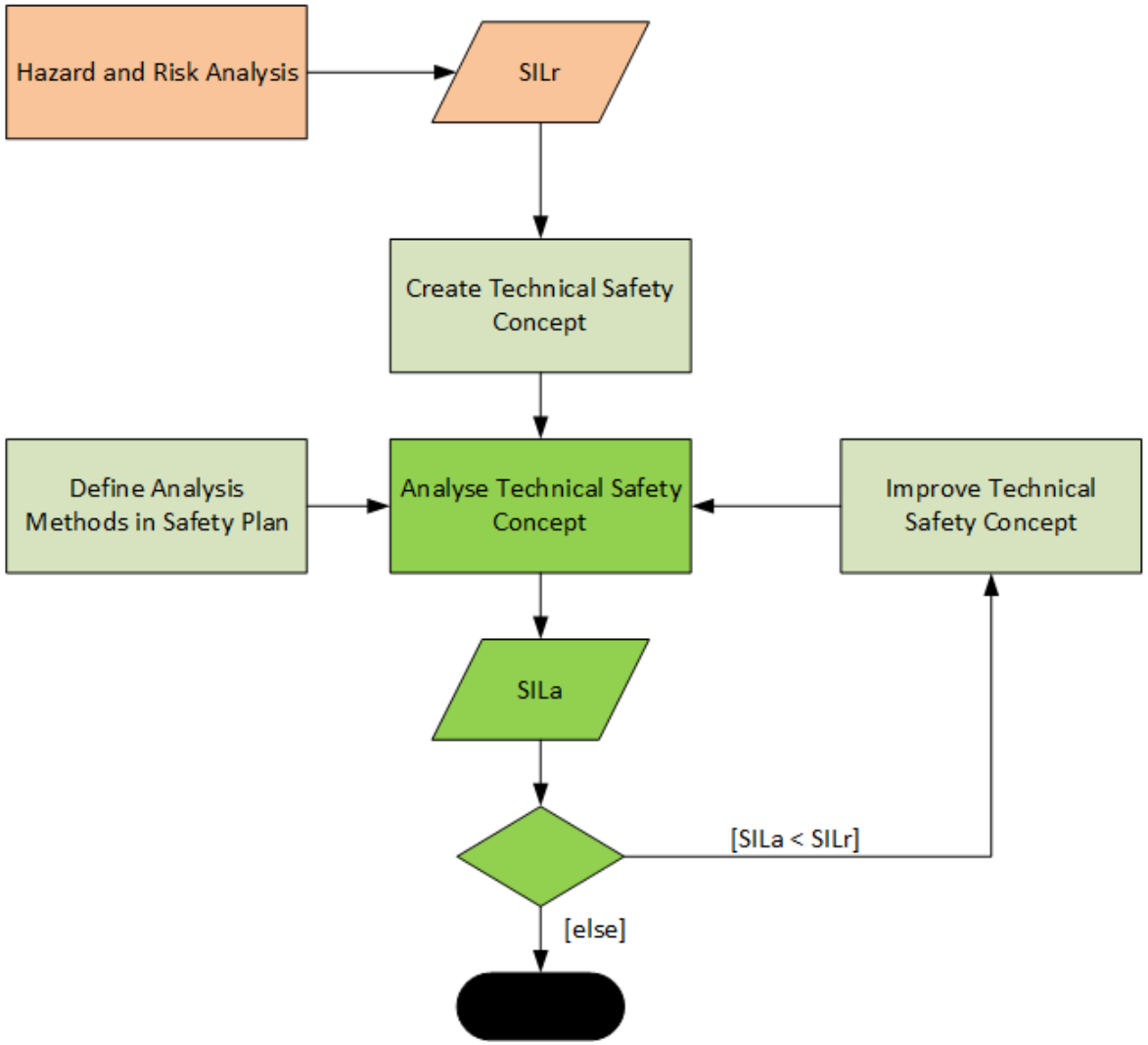


# Safety Analysis Methods

| Method | Description | Comment |
|---|---|---|
| FMEDA (Failure Modes, Effects and Diagnostics Analysis) | Analysis of the effect of random hardware faults on a safety requirement / goal. Including quantitative estimation of failure rates and the resulting probability of a safety goal violation. | • Quantitative<br>• Bottom-Up / Inductive<br>• Hardware |
| FTA (Fault Tree Analysis) | Top level failures are broken down into a Boolean combination of lower level faults (root causes) | • Qualitative / quantitative<br>• Top Down / Deductive<br>• Hardware / Software |
| HAZOP (Hazard and Operability Study) | Structures and systematic analysis of a product, using a keyword based approach (e.g. no, more, less, other than,…) | • Qualitative<br>• Top-Down / Deductive<br>• Mostly for System / Software |
| DFA (Dependent Failure Analysis) | Analysis to identify single events that can cause multiple sub-parts to malfunction. | • Qualitative<br>• Hardware and Software |
| ETA (Event Tree Analysis) | Systematic analysis of the reaction of a system on events. For every event, the probability for a correct handling as well as for an faulty handling are added. | • Quantitative<br>• Top-Down |

# Hardware FMEA

1. FMEA Organization
2. Decomposition of the analyzed item
3. Failure Analysis
4. Risk Evaluation and Mitigation



# FMEA Organisation

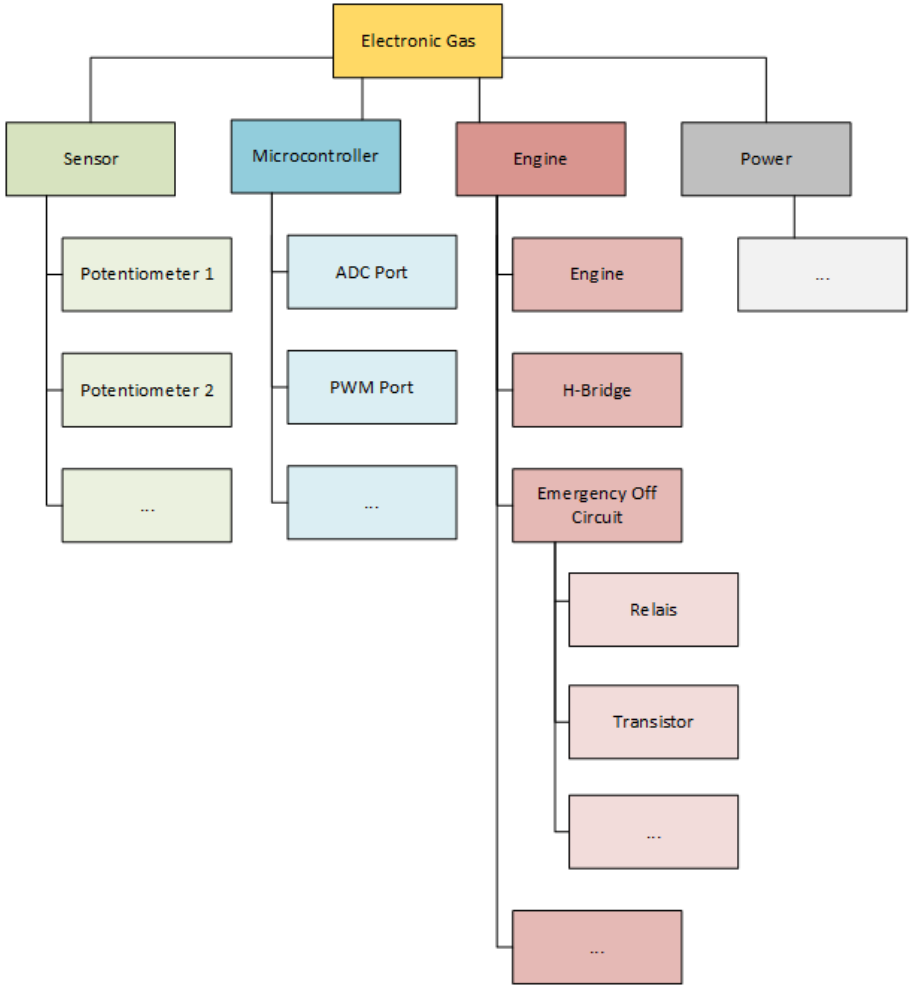
- In order to ensure an efficient FMEA, it is important to clearly identify the item which is going to be analyzed. In safety systems, this either can be the complete safety item (only feasible for small and simple safety items) or an individual safety function.

- A good method for scoping is to identify the components (hardware / mechanical [1]) which implement the safety function. Architectural and Implementation documents describing the implementation of the function as well as technical characteristics of the individual components are collected and a team for performing the FMEA is selected. Furthermore the meetings slots for conducting the FMEA are determined.

- Note 1: In theory, also software can be analyzed using the FMEA methodology, but due to the complexity of software in combination with the more systematic nature of software failures, other methods like static code analysis and reviews are more feasible

# FMEA Item Decomposition

- In the next step, the analyzed item is broken down into smaller systems and components using e.g. a tree method. For a hardware system FMEA, the component level of individual hardware elements like resistors, switches etc. is selected. The following example shows a structure tree (incomplete) of an E-Gas system.



# FMEA Failure Analysis

The analysis is performed in a team as brainstorming. When completing an FMEA, it's important to remember Murphy's Law: "Anything that can go wrong, will go wrong." Participants need to identify all the components, systems, processes and functions that could potentially fail to meet the required level of reliability.

In order to do this in a systematic manner, all leaf elements of the structure tree are transferred into a table, which contains the following data

| Component | Failure Mode | Potential Impact | Severity (B) | Potential Cause | Probability Occurrence (A) | Detection Mode | Probability Detection (E) | Risk Priority Number RPN |
|---|---|---|---|---|---|---|---|---|
| Description of the component | What has gone wrong | What is the potential impact of the failure | Hoe severe is the resulting hazard | What causes the failure to happen | How probable is this potential cause | Can the failure be detected before the hazard happens | How likely is the detection | What is the resulting risk priority |
| Potentiometer P1 | Wrong value from P1 | Wrong target speed calculated, car is out of control | 8 | Pin 1 is disconnected from PCB | 3 | Compare value of P1 against P2 | 1 | 24 |
| Potentiometer P1 | Wrong value from P1 | Wrong target speed calculated, car speed has offset | 7 | Mechanical wear off of sliding contact | 10 | Compare value of P1 against P2. Can only be detected in case of large signal delta. | 2 | **140** |
| ... | | | | | | | | |

There is no general rule on the definition of the severity, occurrence and detection parameters. Typically, a scale from 10 (high risk) down to 1 (low risk) is used. Within the project, a common agreement must be reached how the scale is applied.

# FMEA Risk Evaluation and Mitigation

The Risk priority number is calculated by multiplying the three parameters: RPN = B x A x E
Additional measures are required

- If the RPN value is above an agreed threshold (e.g. 120)
- If B > 8 the failure must be considered as critical
- Furthermore it is recommended to define additional measures if A - E > 3, as this is a failure which happens often but is not detected.

Once the table is completed, one of the team's last steps is to agree on appropriate corrective actions for reducing the occurrence of failure modes, or at least for improving their detection. The FMEA leader should assign responsibility for these actions and set target completion dates. Once corrective actions have been completed, the team should meet again to reassess and rescore the severity, probability of occurrence and likelihood of detection for the top failure modes. This will enable them to determine the effectiveness of the corrective actions taken. These assessments may be helpful in case the team decides that it needs to enact new corrective actions.

# FMEDA – extension of FMEA

Failure modes, effects, and diagnostic analysis (FMEDA) is a systematic analysis technique to obtain quantitative safety goal / system level failure rates, failure modes and diagnostic capabilities. The initial FMEDA added two additional pieces of information to the FMEA analysis process. The first piece of information added in an FMEDA is the quantitative failure data (failure rates and the distribution of failure modes) for all components being analyzed. The second piece of information added to an FMEDA is the probability of the system or subsystem to detect internal failures via automatic on-line diagnostics.

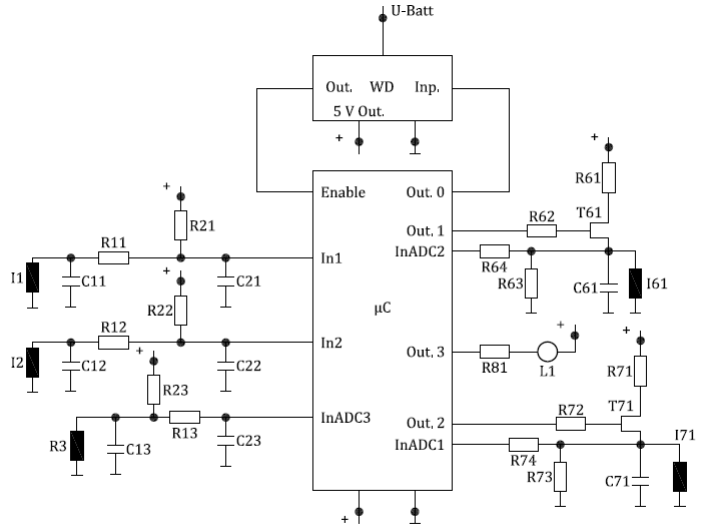



Figure E.1 — Example diagram

# FMEDA Example from ISO26262-5:2018

**Table E.1 — Safety goal 1**

| Component Name | Failure rate/ FIT | Safety-related component to be considered in the calculations? | Failure Mode | Failure mode distribution | Failure mode that has the potential to violate the safety goal in absence of safety mechanisms? | Safety mechanism(s) allowing to prevent the failure mode from violating the safety goal? | Failure mode coverage wrt. violation of safety goal | Residual or Single-Point Fault failure rate/ FIT | Failure mode that may lead to the violation of safety goal in combination with an independent failure of another component? | Detection means? Safety mechanism(s) allowing to prevent the failure mode from being latent? | Failure mode coverage with respect to latent failures | Latent Multiple-Point Fault failure rate/ FIT |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| R3 NOTE 1 | 3 | YES | open | 30 % | X | none | 0 % | 0,9 | | | | |
| | | | closed | 10 % | | | | | | | | |
| | | | drift 0,5 | 30 % | | | | | | | | |
| | | | drift 2 | 30 % | X | | 0 % | 0,9 | | | | |
| R13 NOTE 1, NOTE 2 and NOTE 6 | 2 | YES | open | 90 % | X | none | 0 % | 1,8 | | | | |
| | | | closed | 10 % | X | | 0 % | 0,2 | | | | |
| R23 NOTE 1 | 2 | YES | open | 90 % | | none | | | | | | |
| | | | closed | 10 % | X | | 0 % | 0,2 | | | | |
| C13 NOTE 3 and NOTE 6 | 2 | YES | open | 20 % | X | none | 0 % | 0,4 | | | | |
| | | | closed | 80 % | | | | | | | | |
| C23 NOTE 4 | 2 | NO | open | 20 % | | | | | | | | |
| | | | closed | 80 % | | | | | | | | |
| WD | 20 | YES | Out. Stuck at 1 | 50 % | | | | | X | none | 0 % | 10 |
| | | | Out. Stuck at 0 | 50 % | | | | | | | | |
| T71 NOTE 5 | 5 | YES | open circuit | 50 % | | SM1 | | | | SM1 | | |
| | | | short circuit | 50 % | X | | 90 % | 0,25 | X | | 80 % | 0,45 |
| R71 NOTE 2 and NOTE 6 | 2 | YES | open | 90 % | | | | | | none | | |
| | | | closed | 10 % | | | | | X | | 0 % | 0,2 |
| R72 NOTE 2 and NOTE 6 | 2 | YES | open | 90 % | | | | | | none | | |
| | | | closed | 10 % | | | | | X | | 0 % | 0,2 |

**Table E.1** *(continued)*

| Component Name | Failure rate/ FIT | Safety-related component to be considered in the calculations? | Failure Mode | Failure mode distribution | Failure mode that has the potential to violate the safety goal in absence of safety mechanisms? | Safety mechanism(s) allowing to prevent the failure mode from violating the safety goal? | Failure mode coverage wrt. violation of safety goal | Residual or Single-Point Fault failure rate/ FIT | Failure mode that may lead to the violation of safety goal in combination with an independent failure of another component? | Detection means? Safety mechanism(s) allowing to prevent the failure mode from being latent? | Failure mode coverage with respect to latent failures | Latent Multiple-Point Fault failure rate/ FIT |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| R73 NOTE 4 | 2 | NO | open | 90 % | | | | | | | | |
| | | | closed | 10 % | | | | | | | | |
| R74 NOTE 2 and NOTE 6 | 2 | YES | open | 90 % | | | | | X | none | 0 % | 1,8 |
| | | | closed | 10 % | | | | | X | | 0 % | 0,2 |
| I71 NOTE 4 | 5 | NO | open | 80 % | | | | | | | | |
| | | | closed | 20 % | | | | | | | | |
| C71 NOTE 3 | 2 | YES | open | 20 % | | | | | X | none | | 0,4 |
| | | | closed | 80 % | | | | | | | | |
| R81 NOTE 4 | 2 | NO | open | 90 % | | | | | | | | |
| | | | closed | 10 % | | | | | | | | |
| L1 NOTE 4 | 10 | NO | open | 90 % | | | | | | | | |
| | | | closed | 10 % | | | | | | | | |
| µC | 100 | YES | All | 50 % | X | SM4 | 90 % | 5 | X | SM4 | 100 % | 0 |
| | | | All | 50 % | | | | | | | | |
| | | | | | | | ∑ 9,65 | | | | ∑ 13,25 | |

Total failure rate 163 FIT
Total Safety-Related 142 FIT
Total Non Safety-Related 21 FIT

Single-Point Fault Metric = 1-(9,65/142) = 93,2 %

Latent Fault Metric = 1-(13,25/(142-9,65)) = 90,0 %

# Software: Which parts do we have to qualify using which methods?

**Table 7 — Methods for software unit verification**

| | Methods | ASIL A | ASIL B | ASIL C | ASIL D |
|---|---|---|---|---|---|
| 1a | Walk-through[a] | ++ | + | o | o |
| 1b | Pair-programming[a] | + | + | + | + |
| 1c | Inspection[a] | + | ++ | ++ | ++ |
| 1d | Semi-formal verification | + | + | ++ | ++ |
| 1e | Formal verification | o | o | + | + |
| 1f | Control flow analysis[b, c] | + | + | ++ | ++ |
| 1g | Data flow analysis[b, c] | + | + | ++ | ++ |
| 1h | Static code analysis[d] | ++ | ++ | ++ | ++ |
| 1i | Static analyses based on abstract interpretation[e] | + | + | + | + |
| 1j | Requirements-based test[f] | ++ | ++ | ++ | ++ |
| 1k | Interface test[g] | ++ | ++ | ++ | ++ |
| 1l | Fault injection test[h] | + | + | + | ++ |
| 1m | Resource usage evaluation[i] | + | + | + | ++ |
| 1n | Back-to-back comparison test between model and code, if applicable[j] | + | + | ++ | ++ |

Reviews / Analytical Qualification

**Table 9 — Structural coverage metrics at the software unit level**

| | Methods | ASIL A | ASIL B | ASIL C | ASIL D |
|---|---|---|---|---|---|
| 1a | Statement coverage | ++ | ++ | + | + |
| 1b | Branch coverage | + | ++ | ++ | ++ |
| 1c | MC/DC (Modified Condition/Decision Coverage) | + | + | + | ++ |

Code Coverage

**Table 8 — Methods for deriving test cases for software unit testing**

| | Methods | ASIL A | ASIL B | ASIL C | ASIL D |
|---|---|---|---|---|---|
| 1a | Analysis of requirements | ++ | ++ | ++ | ++ |
| 1b | Generation and analysis of equivalence classes[a] | + | ++ | ++ | ++ |
| 1c | Analysis of boundary values[b] | + | ++ | ++ | ++ |
| 1d | Error guessing based on knowledge or experience[c] | + | + | + | + |

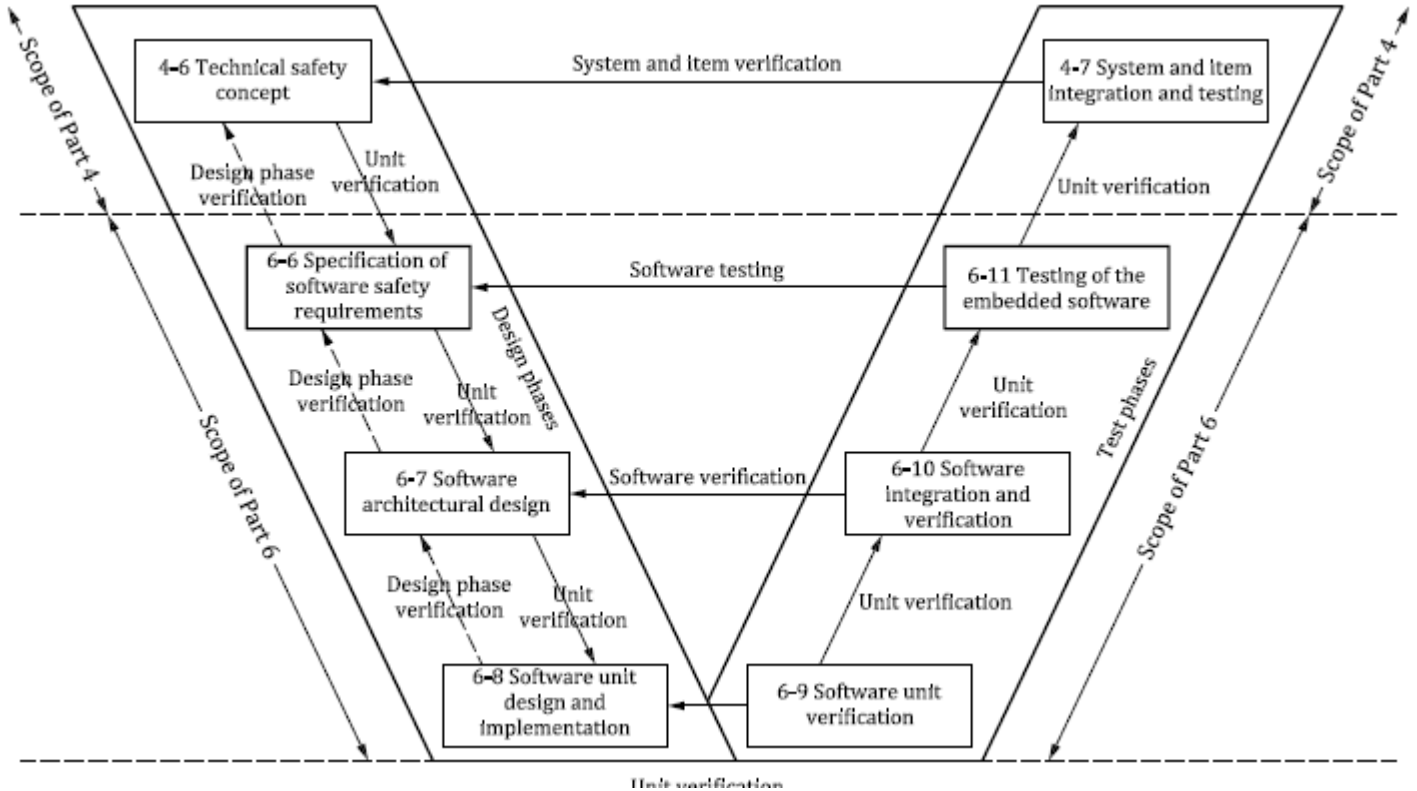
[a] Equivalence classes can be identified based on the division of inputs and outputs, such that a representative test value can be selected for each class.

[b] This method applies to interfaces, values approaching and crossing the boundaries and out of range values.

[c] Error guessing tests can be based on data collected through a "lessons learned" process and expert judgment.

Test Specification

# Test on various levels

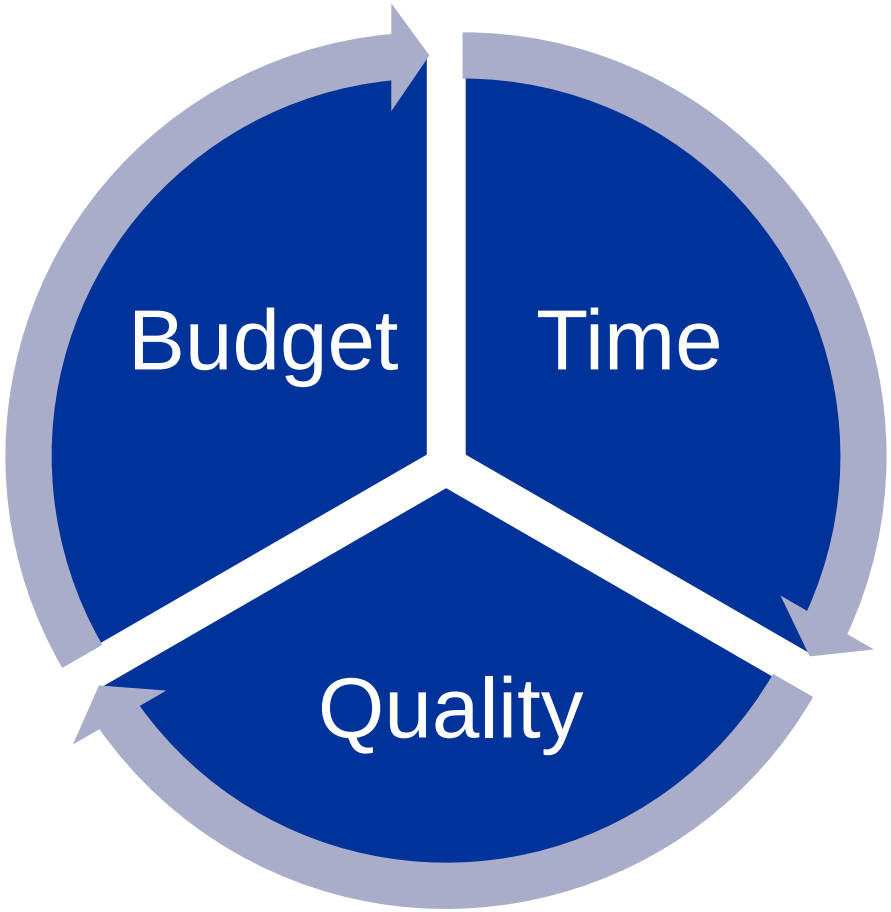


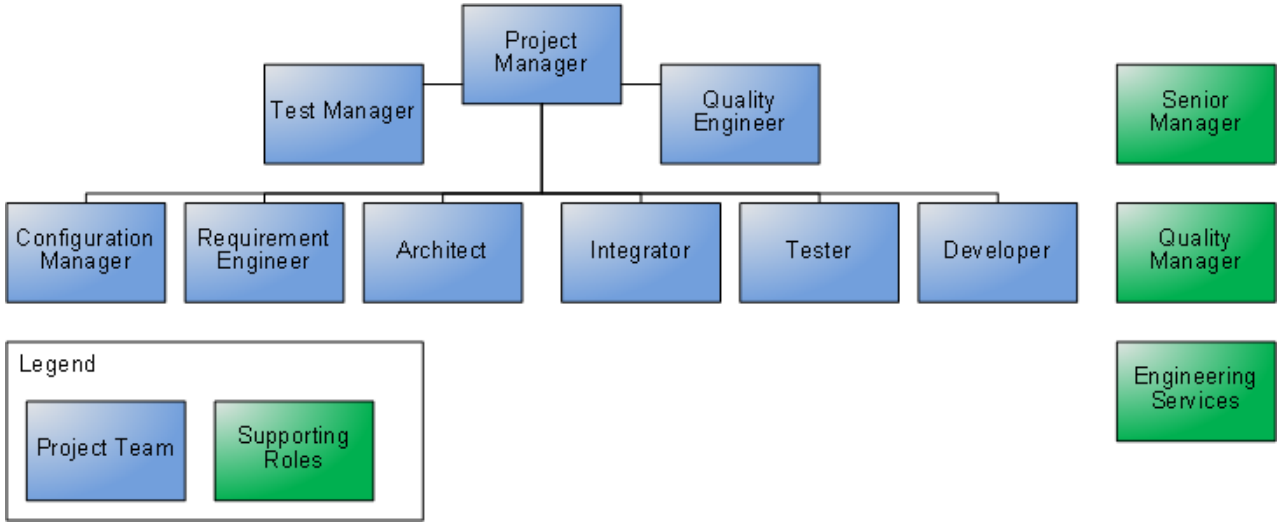

Complete function, ILO cycle

Interfaces, resources (memory, time)

Detailed test of the unit (class)

# We always will have this conflict!



# Problem: Safety Bias

*„People will focus and interpret evidence in a way that confirms the goal they have set for themselves. If the goal is to prove the system is safe, they will focus on the evidence that shows it is safe and create an argument for safety. If the goal is to show that the system in unsafe, the evidence used and the interpretation of available evidence will be quite different. People also tend to interpret ambiguous evidence as supporting their existing position."*

C. Hobbs

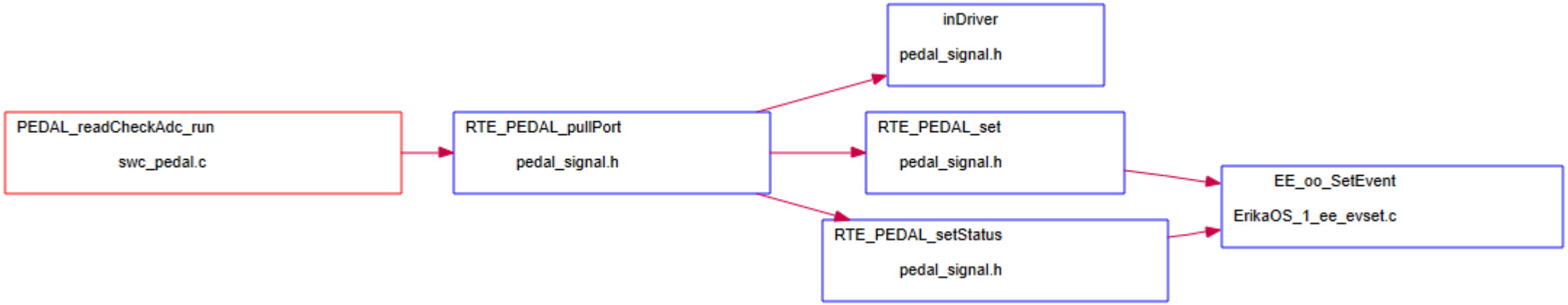
Developer and Tester are not the same person!

# Traceability

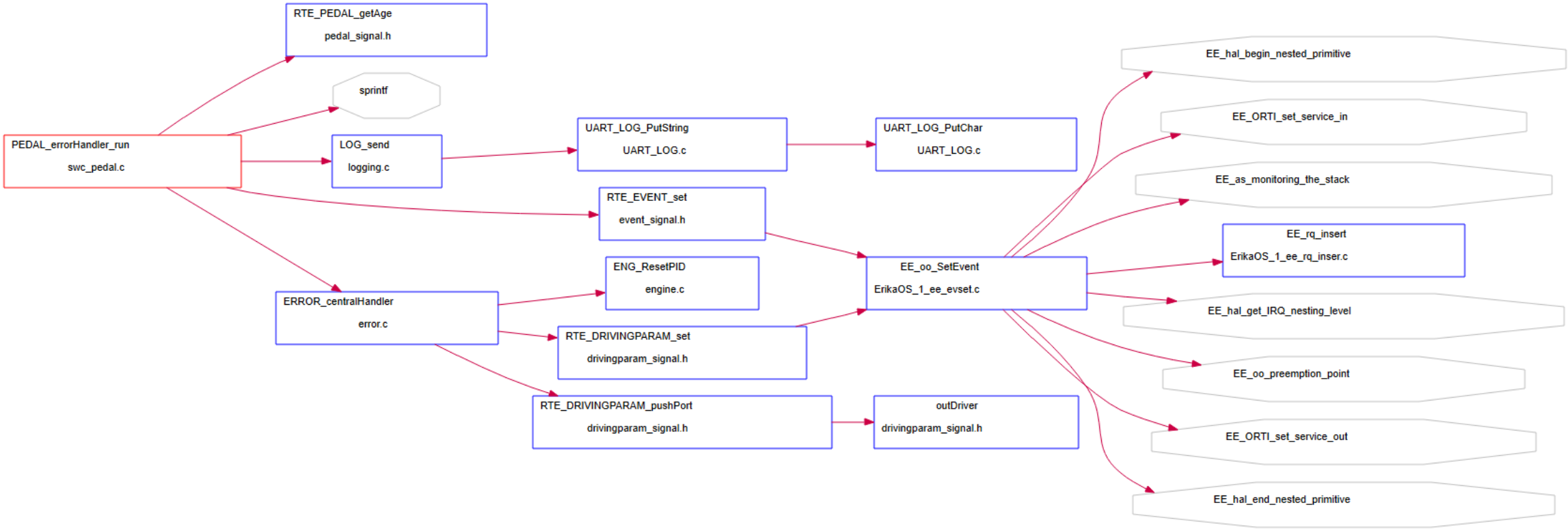
| Id | Safety Goal | $ASIL_{req}$ | Implementation Unit |
|---|---|---|---|
| SG1 | The gas pedal signal must be reliable | B | • void PEDAL_readCheckAdc_run()<br>• void PEDAL_errorHandler_run() |
| SG2 | The brake button signal must be reliable | D | |
| SG3 | Engine current speed must be in line with the control speed | B | |
| SG4 | Calculated control values must be correct | D | |
| SG5 | State changes may only be performed if car is stopped | D | |
| SG6 | Avoid starting the engine without a clear driver interaction | QM | |
| SG7 | Limit reverse speed | QM | |

# Getting an overview of dependencies
# Pedal Read Function

Note: function pointers are not shown!



# Getting an overview of dependencies
# Pedal Error Handler



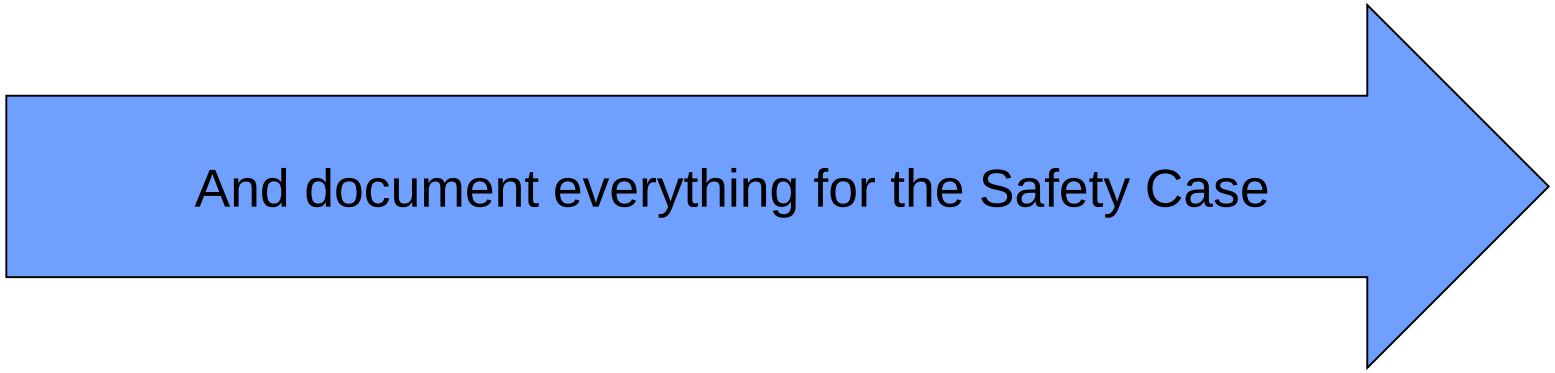
# Qualifying Pedal

The pedal read function is a ASIL B function, i.e. we must

- Review: Inspection
- Static Code Analysis
- Requirements Based Test (Equivalence, Boundary)
- Interface Based Test
- Proof statement and branch coverage

We should

- Fault injection test (as we want to identify potential faults)



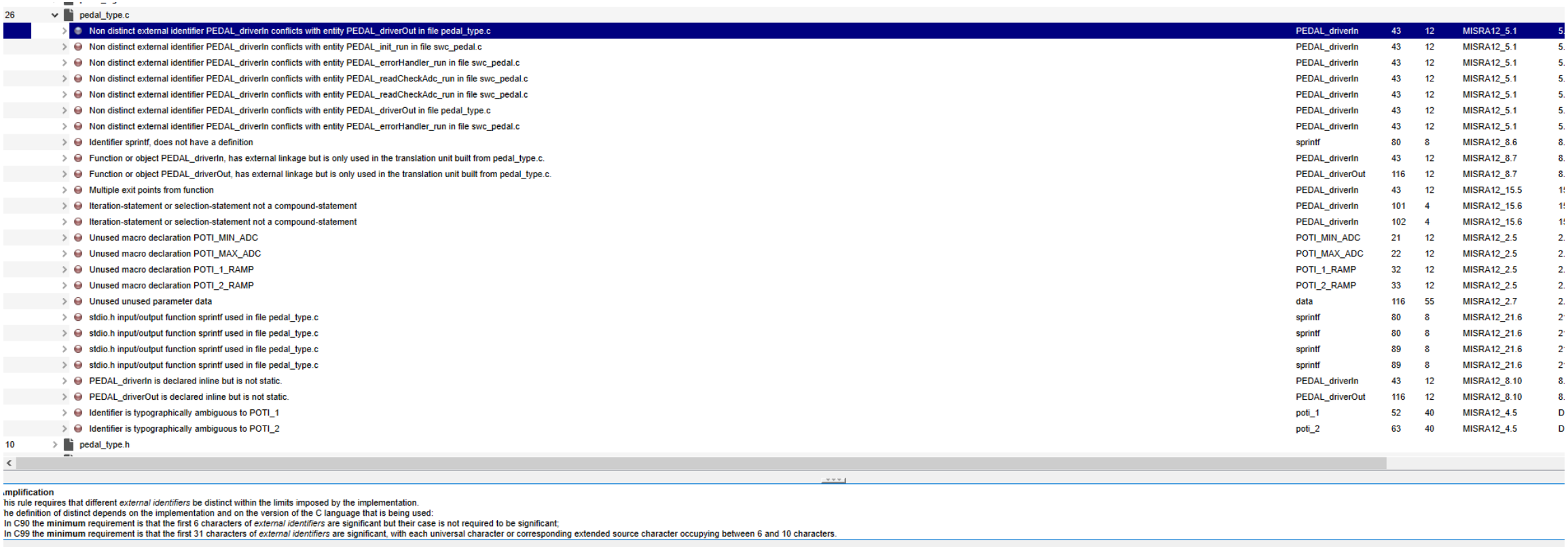
# Review Organisation

- The code (design) to be reviewed in sent to the experts together with a review minute document
- The reviewers document their finding (either in the review document or informally)
- In a review meeting the findings are collected
- The document owner checks the findings and decides on an improvement, which is documented in the review minute document
- The fixed code together with the final review document is sent to the review participants for information

# Example review comments

| Author | Location | Finding | Improvement |
|---|---|---|---|
| Fromm | SWC_pedal.c<br>Line 29 | Return value is ignored | |
| Fromm | SWC_pedal.c<br>Line 34 | In case the time base is changed, the critical age of the signal must be modified. This might be forgotten, as it is hidden in the implementation file | |
| Fromm | SWC_pedal.c<br>Line 49 and 51 | In line 49, we fire an event to the control state machine. In line 51, we call the central error handler, which sets another signal (so_drivingparam_signal), which will also be used by the control state machine. Possible race! | |
| | | … | |
| | | | |

# Static Code Analysis



| | | | | | | |
|---|---|---|---|---|---|---|
| 26 | pedal_type.c | | | | | |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_driverOut in file pedal_type.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_init_run in file swc_pedal.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_errorHandler_run in file swc_pedal.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_readCheckAdc_run in file swc_pedal.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_readCheckAdc_run in file swc_pedal.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_driverOut in file pedal_type.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Non distinct external identifier PEDAL_driverIn conflicts with entity PEDAL_errorHandler_run in file swc_pedal.c | PEDAL_driverIn | 43 | 12 | MISRA12_5.1 | 5. |
| | Identifier sprintf, does not have a definition | sprintf | 80 | 8 | MISRA12_8.6 | 8. |
| | Function or object PEDAL_driverIn, has external linkage but is only used in the translation unit built from pedal_type.c. | PEDAL_driverIn | 43 | 12 | MISRA12_8.7 | 8. |
| | Function or object PEDAL_driverOut, has external linkage but is only used in the translation unit built from pedal_type.c. | PEDAL_driverOut | 116 | 12 | MISRA12_8.7 | 8. |
| | Multiple exit points from function | PEDAL_driverIn | 43 | 12 | MISRA12_15.5 | 1! |
| | Iteration-statement or selection-statement not a compound-statement | PEDAL_driverIn | 101 | 4 | MISRA12_15.6 | 1! |
| | Iteration-statement or selection-statement not a compound-statement | PEDAL_driverIn | 102 | 4 | MISRA12_15.6 | 1! |
| | Unused macro declaration POTI_MIN_ADC | POTI_MIN_ADC | 21 | 12 | MISRA12_2.5 | 2. |
| | Unused macro declaration POTI_MAX_ADC | POTI_MAX_ADC | 22 | 12 | MISRA12_2.5 | 2. |
| | Unused macro declaration POTI_1_RAMP | POTI_1_RAMP | 32 | 12 | MISRA12_2.5 | 2. |
| | Unused macro declaration POTI_2_RAMP | POTI_2_RAMP | 33 | 12 | MISRA12_2.5 | 2. |
| | Unused unused parameter data | data | 116 | 55 | MISRA12_2.7 | 2. |
| | stdio.h input/output function sprintf used in file pedal_type.c | sprintf | 80 | 8 | MISRA12_21.6 | 2! |
| | stdio.h input/output function sprintf used in file pedal_type.c | sprintf | 80 | 8 | MISRA12_21.6 | 2! |
| | stdio.h input/output function sprintf used in file pedal_type.c | sprintf | 89 | 8 | MISRA12_21.6 | 2! |
| | stdio.h input/output function sprintf used in file pedal_type.c | sprintf | 89 | 8 | MISRA12_21.6 | 2! |
| | PEDAL_driverIn is declared inline but is not static. | PEDAL_driverIn | 43 | 12 | MISRA12_8.10 | 8. |
| | PEDAL_driverOut is declared inline but is not static. | PEDAL_driverOut | 116 | 12 | MISRA12_8.10 | 8. |
| | Identifier is typographically ambiguous to POTI_1 | poti_1 | 52 | 40 | MISRA12_4.5 | D |
| | Identifier is typographically ambiguous to POTI_2 | poti_2 | 63 | 40 | MISRA12_4.5 | D |
| 10 | pedal_type.h | | | | | |

**mplification**
his rule requires that different *external identifiers* be distinct within the limits imposed by the implementation.
he definition of distinct depends on the implementation and on the version of the C language that is being used:
In C90 the **minimum** requirement is that the first 6 characters of *external identifiers* are significant but their case is not required to be significant;
In C99 the **minimum** requirement is that the first 31 characters of *external identifiers* are significant, with each universal character or corresponding extended source character occupying between 6 and 10 characters.

# Static code analysis

Similar like review comments, the results of a static code analysis are recommendations, which must be checked and decided.

| Finding | Evaluation | Decision |
|---|---|---|
| 5.1 External identifiers shall be distinct | This violates our namespace concept. In case a (old) compiler truly requires a distinction in the first 6 characters, an error will be thrown. | Ignored |
| 8.7 Function should not be external, if it is only called inside the file | Wrong analysis result, the function pointer is not evaluated correctly. | Ignored |
| 2.5 Unused macro definition | Some macros are not used (anymore) | Check with other files and clean up |

# Review versus Static Code Analysis

## Review

- Strong on conceptional findings
- Comparable high efforts

Combination makes sense!

## Static Code Analysis

- Strong on finding “many small issues” and “legacy” problems
- Cheap

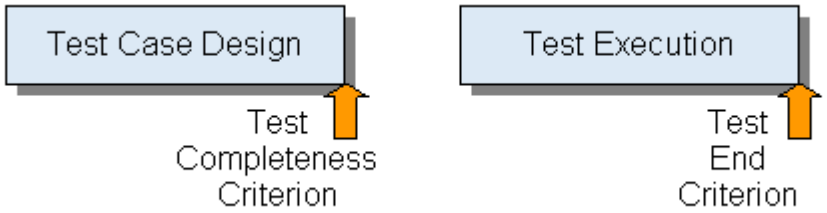
**Rule 5.7 (advisory): No identifier name should be reused.**

Regardless of scope, no identifier should be re-used across any files in the system. This rule incorporates the provisions of Rules 5.2, 5.3, 5.4, 5.5 and 5.6.

```
struct air_speed
{
   uint16_t speed;    /* knots */
} * x;
struct gnd_speed
{
   uint16_t speed;    /* mph                                          */
                      /* Not Compliant - speed is in different units */
} * y;
x->speed = y->speed;
```

Where an identifier name is used in a header file, and that header file is included in multiple source files, this rule is not violated. The use of a rigorous naming convention can support the implementation of this rule.

# Testing




## Test Completeness

- Criteria to measure the completeness of the specified testcases, e.g.
- All requirements are tested
- All relevant signal value combinations have been tested
- A C1 coverage of 95% has been reached

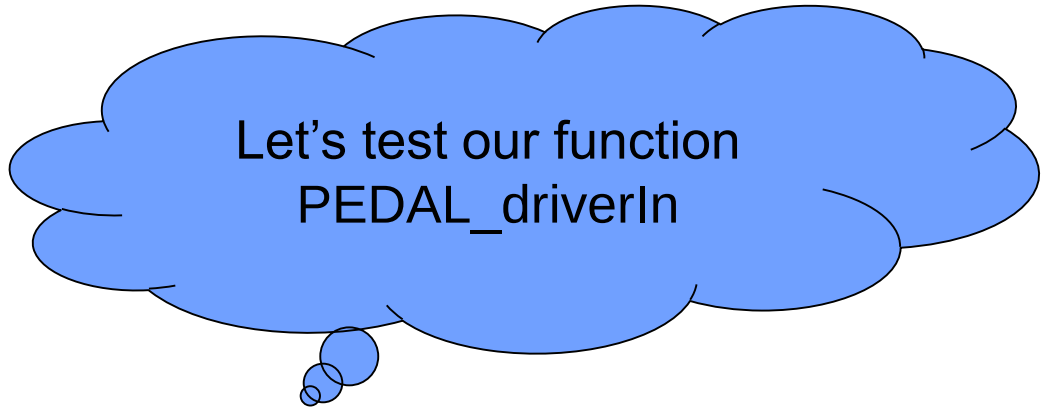
## Test End

- Have sufficient tests been executed (based on the identified risk assessment)
- Is the result of the test sufficient to release the system, e.g.
- No testcases for safety functions failed
- Regression test: All tests related to the modified function AND all safety testcases AND all integration tests AND all relevant system tests have been executed.

# Black Box Testing


Based on

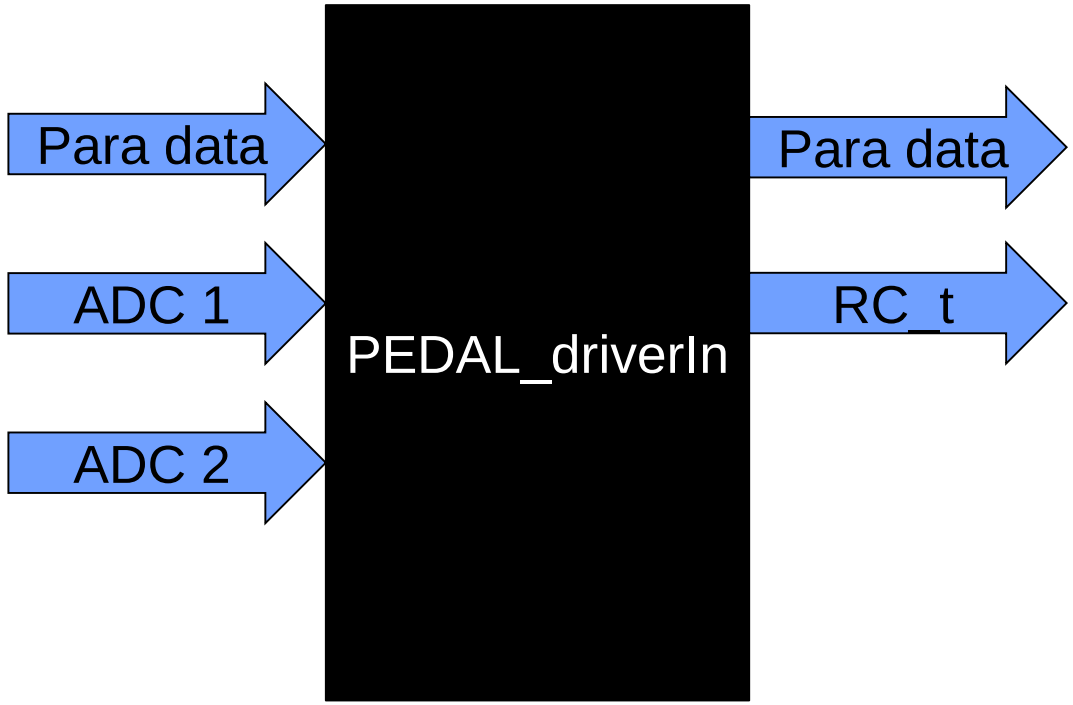
- Requirements
- Interface
- Experience / prior tests
- Architecture
- …



# PEDAL_driverIn

Requirements

- Calculate the average value of the 2 potentiometers
- In case of a short circuit of broken connection an error will be thrown
- In case of strongly deviating values an error will be thrown
- Errors will be detected based on configuration values, i.e. we must test the logic AND the parameters




Are the requirements specific enough to derive testcases?

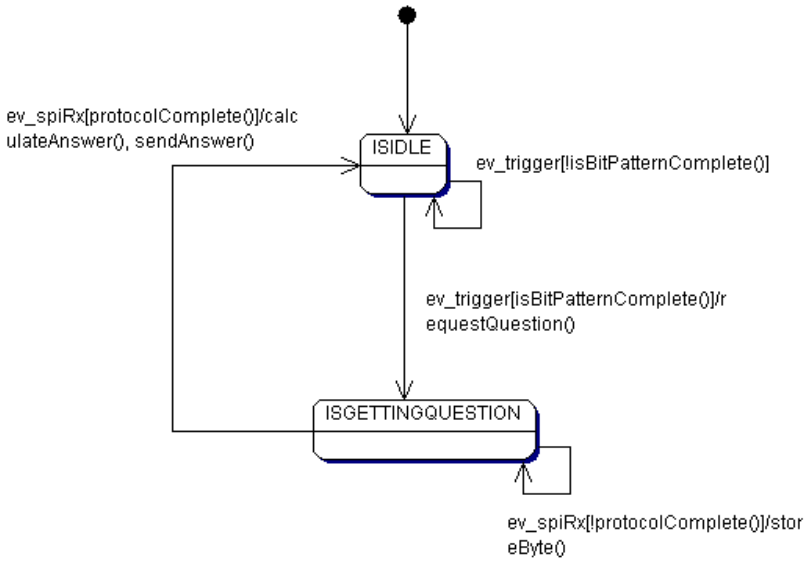
# Testcases (equivalence classes)

| Id | Description | Input | Expected Result | Result |
|---|---|---|---|---|
| 1.1 | Normal behavior – middle position | • All connections valid<br>• Potentiometer in middle position | • Average value of both potentiometers, i.e. 128 +/- 10<br>• Signal valid<br>• RC_SUCCESS | |
| 1.2 | Normal behavior – right position | • All connections valid<br>• Potentiometer in right position | • Average value of both potentiometers, i.e. 0..10<br>• Signal valid<br>• RC_SUCCESS | |
| 1.3 | Normal behavior – left position | • All connections valid<br>• Potentiometer in left position | • Average value of both potentiometers, i.e. 245..255<br>• Signal valid<br>• RC_SUCCESS | |
| 1.4 | GND broken | • GND of potentiometer disconnected<br>• Potentiometer in middle position | • Previous signal value obtained<br>• Signal invalid<br>• RC_ERRORRANGE | |
| 1.5 | … | | | |

# Testcases (boundary values - hardware)

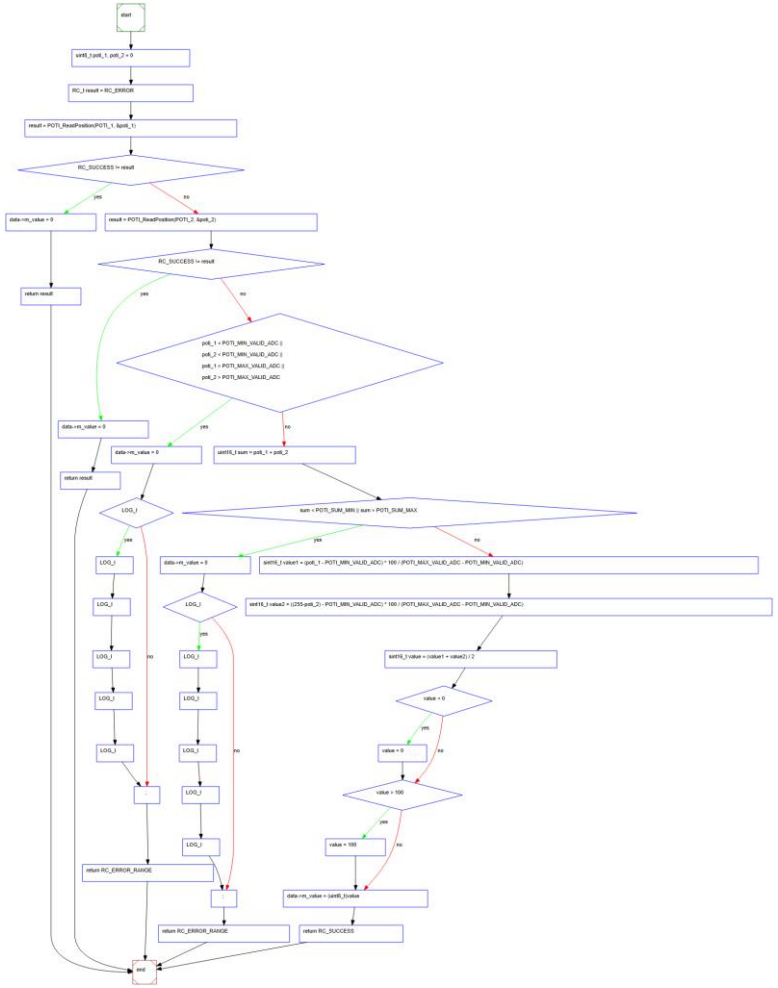
| Id | Description | Input | Expected Result | Result |
|---|---|---|---|---|
| 2.1 | Normal behavior – right position | • All connections valid<br>• Potentiometer in right position<br>• Slow movement out of right position to middle | • Average value of both potentiometers, i.e. 0..10<br>• Signal valid<br>• RC_SUCCESS<br>• Potentiometer values changes immediately | |
| 2.2 | Normal behavior – left position | • All connections valid<br>• Potentiometer in left position<br>• Slow movement out of left position to middle | • Average value of both potentiometers, i.e. 245..255<br>• Signal valid<br>• RC_SUCCESS<br>• Potentiometer values changes immediately | |
| 2.3 | … | | | |

# State Machine Testing – Example Q/A Watchdog




| | ISIDLE | ISGETTINGQUESTION |
|---|---|---|
| ev_trigger[isBitPatternComplete()] | requestQuestion() / ISGETTINGQUESTION | -/- |
| ev_trigger[!isBitPatternComplete()] | -/- | -/- |
| ev_spiRX[protocolComplete()] | -/- | calculateAnswer(), sendAnswer / ISIDLE |
| ev_spiRX[!protocolComplete()] | -/- | storeByte() / - |

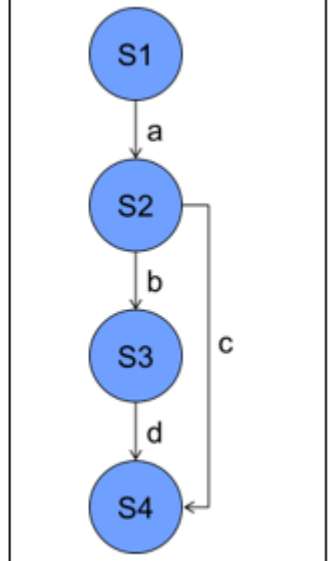
# Whitebox Testing: Analysing the code structure to derive (additional) testcases



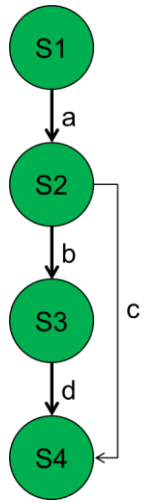
- Creating the activity diagram (control flow) helps you to create relevant testcases based on the structure

# Coverage Test – Statement C0 and Branch C1

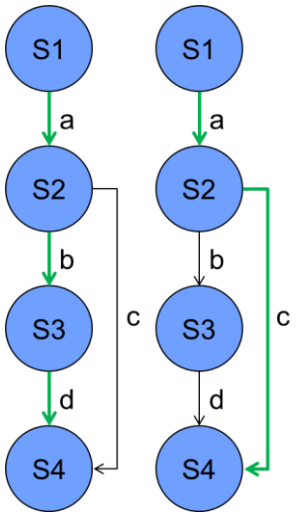
```
b = getValue();      //Statement s1


if (b > 10)          //Statement s2


{
      b = 10;        //Statement s3
}


setValue(b);         //Statement s4
```




C0




C1



# Comparison

## C0 – Statement Coverage

- Detection of “dead code”
- Simple implementation
- Supported by some debuggers “out of the box”

- Empty paths are ignored
- Data values are ignored
- Multiple condition logic is ignored

## C1 – Branch Coverage

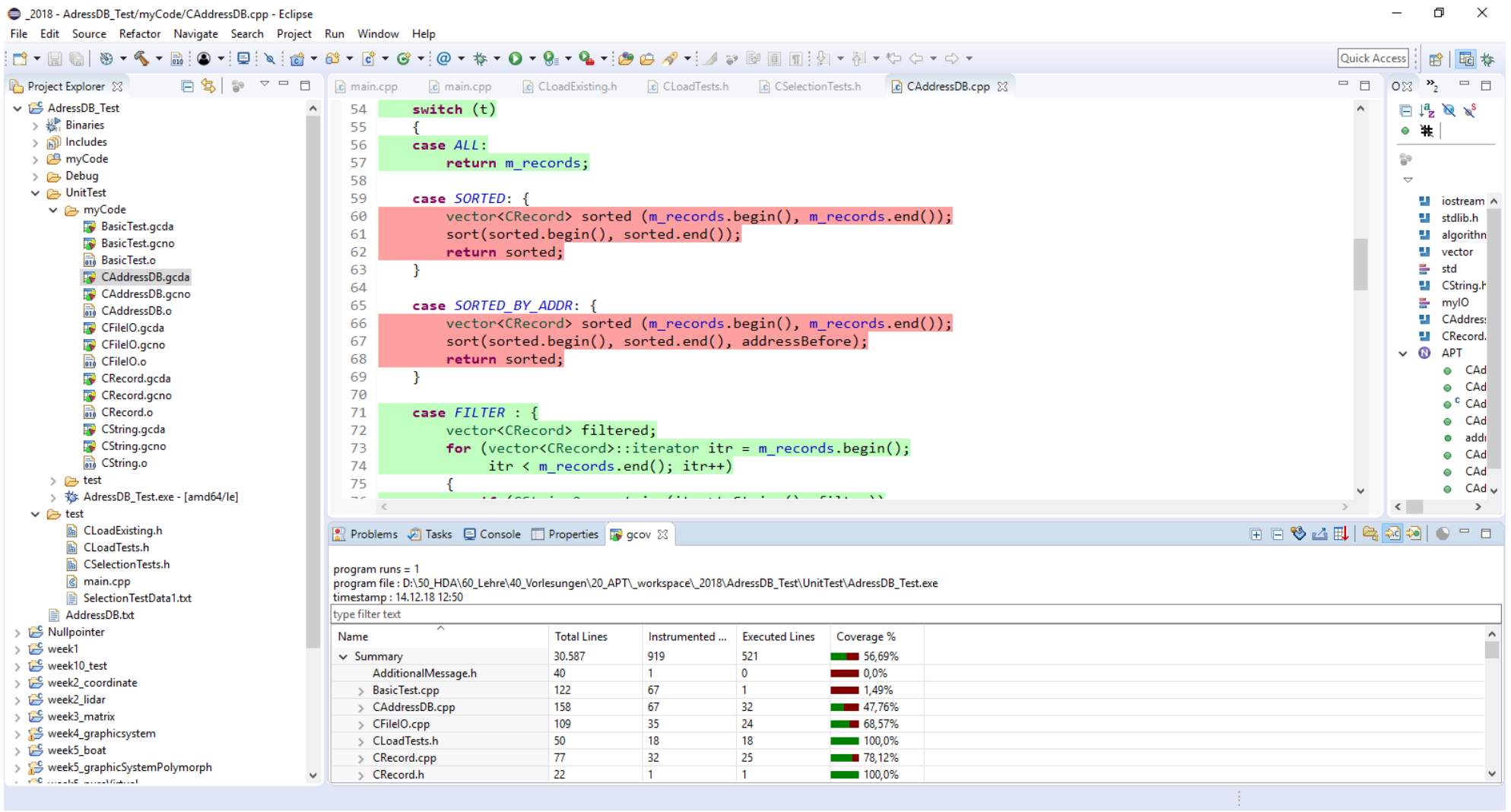
- Checks for all branches
- Better representation of program logic
- Code instrumentation required

- Data values are ignored
- Multiple condition logic is ignored

## MC/DC – Modified Condition / Decision Coverage

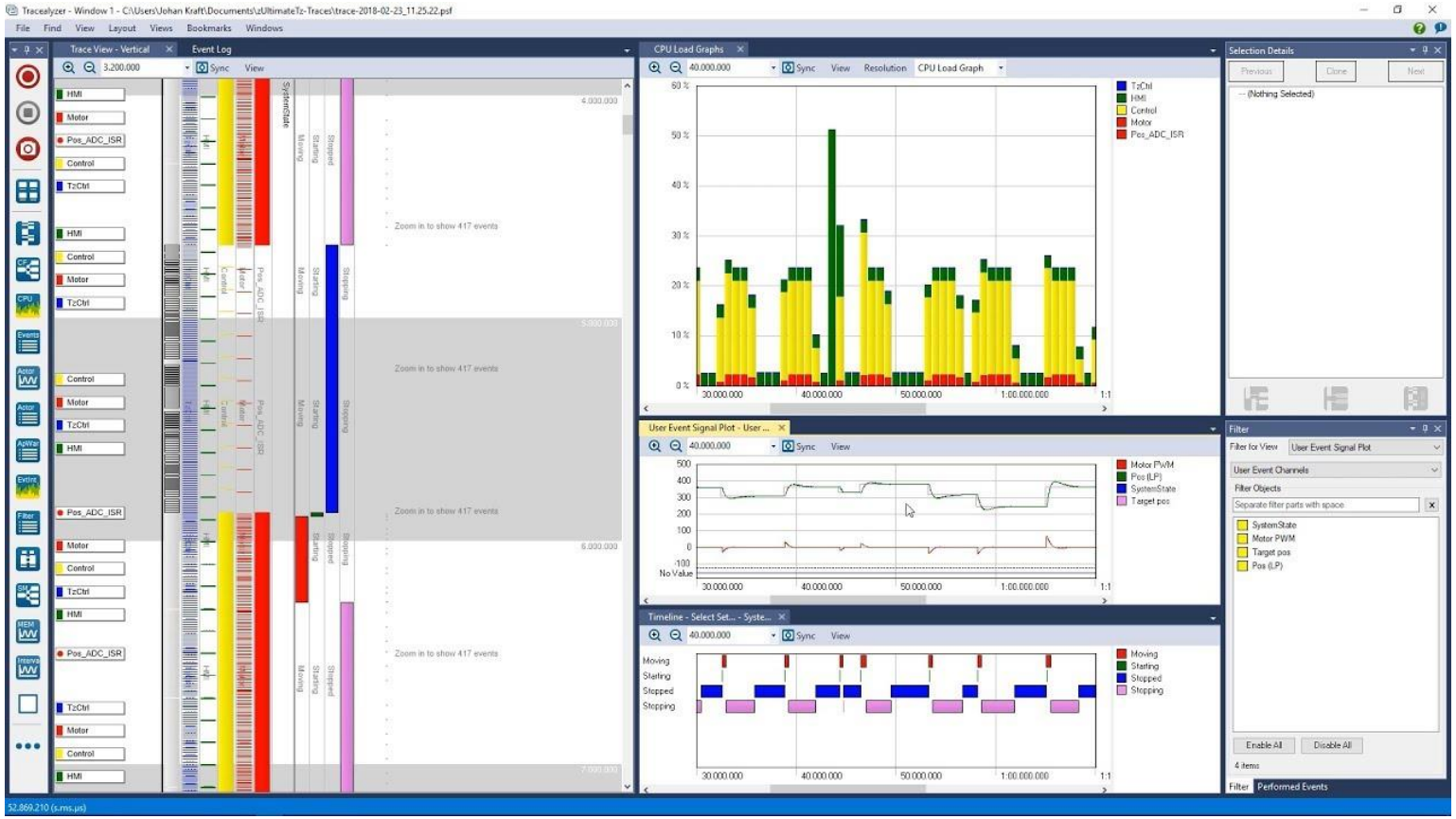
- Adds a testcase for every Boolean condition
- Data values are ignored

# Example GCOV



# Integration Test

- Interface test (do the functions “fit”)
- Resource Test
  - Memory
    - Global RAM/Flash
    - Stack (per task)
    - EEPROM
  - CPU
    - Time behavior



# System Test

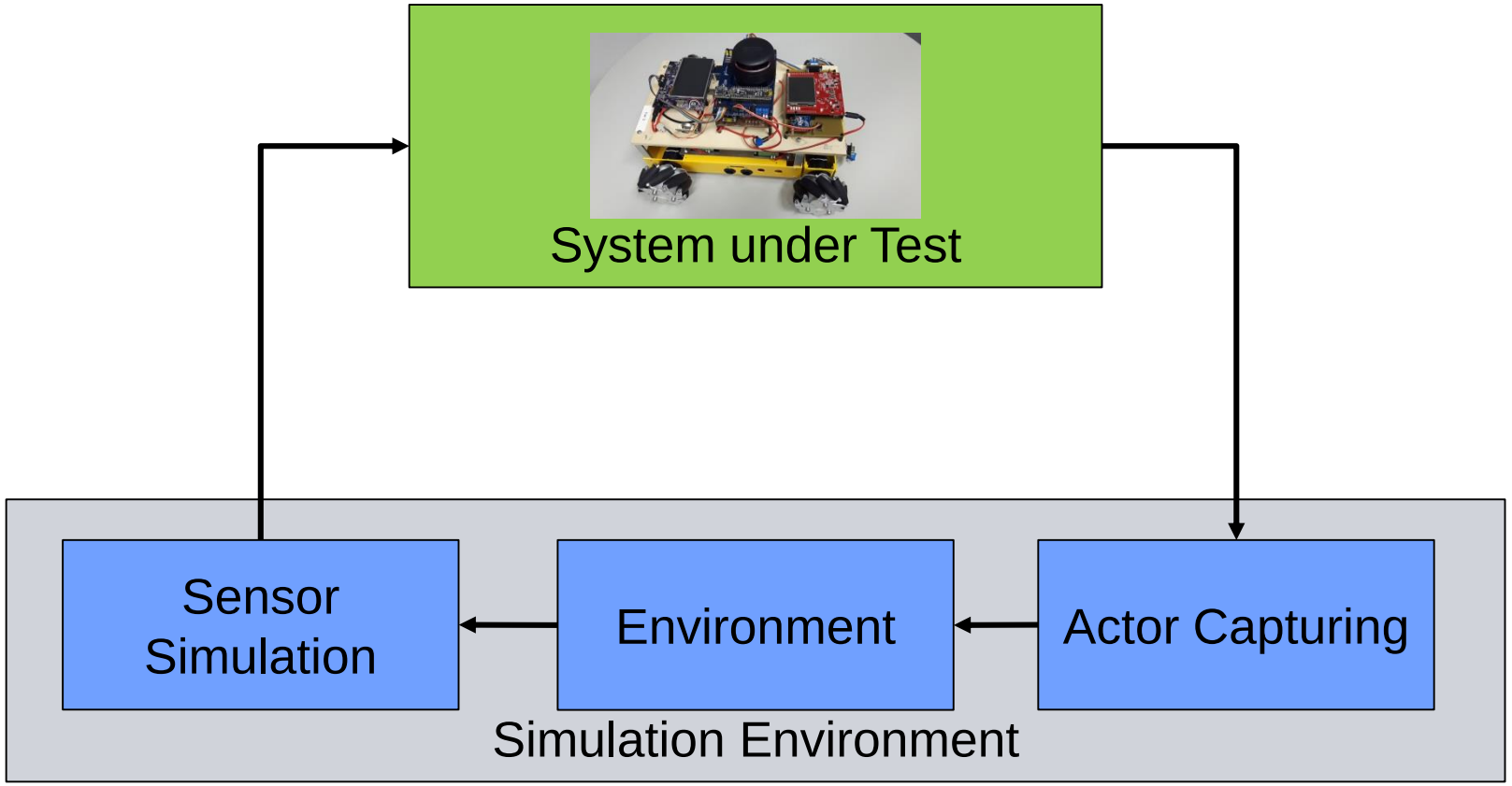
**Table 13 — Test environments for conducting the software testing**

| Methods | | ASIL | | | |
|---|---|---|---|---|---|
| | | A | B | C | D |
| 1a | Hardware-in-the-loop | ++ | ++ | ++ | ++ |
| 1b | Electronic control unit network environments[a] | ++ | ++ | ++ | ++ |
| 1c | Vehicles | + | + | ++ | ++ |

[a] Examples include test benches partially or fully integrating the electrical systems of a vehicle, "lab-cars" or "mule" vehicles, and "rest of the bus" simulations.

**11.4.2** The testing of the embedded software shall be performed using methods as listed in Table 14 to provide evidence that the embedded software fulfils the software requirements as required by their respective ASIL.

# HIL/SIL


System under Test
Sensor Simulation
Environment
Actor Capturing
Simulation Environment

# Wrap-Up

- Hazard and Risk Analysis
- Definition of Safety Goals and Safety Requirements

- Creation of a System Architecture (Functional Model)
- Selection of reliable hardware, increasing hardware diagnostics
- Implementation of safety functions in software (hardware and user fault detection)

- Application of coding standards
- Systematic qualification and test

# What you should take from this lecture

- Safety plays a more and more prominent role for embedded systems
- Algorithms become more complex, the machine takes over more and more descisions from humans

- You need to carefully identify potential risks
- Develop a solid technical concept
- And verify this independently

- You need time – a safety project requires much more time and resources compared to a normal project
- You have to prove your safety case!
- Hoping that your system might work is not enough anymore!

# The simple-most solution

```
int state = 0;
while(1)
{
        if (state == 0 && Pin_Ignition_Read() == 1) state = 1;
        if (state == 1 && Pin_Brake_Read() == 1) state = 2;
        if (state == 2 && Pin_Brake_Read() == 0) state = 3;

        uint8_t speed = ADC_Read();
        if (state == 3 && Pin_Brake_Read() == 0) PWM_write(speed); else PWM_write(0);
}
```


SAFETY
FIRST